REMOTE ROBOTIC SURVEILLANCE SYSTEM

A Mini Project Report submitted to

MOHAN BABU UNIVERSITY

in Partial Fulfillment of the Requirements for the Award of the degree of

BACHELOR OF TECHNOLOGY
IN
COMPUTER SCIENCE AND ENGINEERING (CYBER SECURITY)

Submitted by

PATHANGE PRAVEEN KUMAR	**(25102A020008)**
M.GANGA LALLU LEEDHAR	**(25102A020032)**

Under the Guidance of

Mr. P. Yogendra Prasad
Assistant Professor
Department of Data Science



Department of Data Science

SCHOOL OF COMPUTING

MOHAN BABU UNIVERSITY

Sree Sainath Nagar, Tirupati – 517 102, A.P., INDIA

2026-27

MOHAN BABU UNIVERSITY

Sree Sainath Nagar, Tirupati – 517 102, A.P., INDIA
2026-27

DEPARTMENT OF DATA SCIENCE

CERTIFICATE

This is to certify that the mini project report entitled

"REMOTE ROBOTIC SURVEILLANCE SYSTEM"

is the Bonafide work done by

PATHANGE PRAVEEN KUMAR	**(25102A020008)**
M.GANGA LALLU LEEDHAR	**(25102A020032)**

in the Department of **Data Science**, and submitted to Mohan Babu University, Tirupati in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Cyber Security) during the academic year 2026-27. This work has beencarried out under my supervision. The results of this mini project work havenot been submitted to any university for the award of any degree or diploma.

Guide:

Mr. P. Yogendra Prasad
Assistant Professor
Dept. of Data Science

Head:

Dr. K. Reddy Madhavi
Professor & Head
Dept. of Data Science

INTERNAL EXAMINER

EXTERNALEXAMINER

DEPARTMENT OF DATA SCIENCE

Vision

To become a Centre of Excellence in Data Science by imparting high quality education through teaching, training and research

Mission

- To impart quality education in Computer Science and Engineering with specializations in Data Science by disseminating knowledge through contemporary curriculum, competent faculty and effective teaching-learning methodologies.
- Nurture research, innovation and entrepreneurial skills among students and faculty to contribute to the needs of industry and society.
- Inculcate professional attitude, ethical and social responsibilities for prospective and promising Engineering profession.
- Encourage students to engage in life-long learning by creating awareness of the contemporary developments in Computer Science and Engineering with specialization in Data Science.

PROGRAM EDUCATIONAL OBJECTIVES

After few years of graduation, the graduates of B. Tech. CSE(CS) will:

PEO1. Pursue higher studies in Computer Science, Data science or Management.

PEO2. Become successful entrepreneurs or be employed by acquiring required skill sets in the domains of Cyber Security and allied areas.

PEO3. Exhibit progression and effective adaptation to technological developments through life-long learning to address ever changing industrial requirements and follow ethical attitude in professional practice.

PROGRAM SPECIFIC OUTCOMES

On successful completion of the Program, the graduates of B. Tech. CSE(CS) program will be able to:

PSO1. Apply appropriate data analytical techniques for building effective decision-making systems.

PSO2. Develop intelligent systems using novel Machine Learning and Artificial Intelligence techniques.

PSO3. Design and develop efficient software systems using modern tools, techniques, and platforms to meet societal needs.

PSO4. Apply suitable tools and techniques to build secure distributed systems.

PROGRAM OUTCOMES

On successful completion of the Program, the graduates of B.Tech. CSE (CS) Program will be able to:

PO1. **Engineering knowledge**: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2. **Problem analysis**: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3. **Design/development of solutions**: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4. **Conduct investigations of complex problems**: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.

PO5. **Modern tool usage**: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6. **The engineer and society**: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7. **Environment and sustainability**: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8. **Ethics**: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9. **Individual and team work**: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10. **Communication**: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11. **Project management and finance**: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12. **Life-long learning**: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

DECLARATION

We hereby declare that this project report titled **"REMOTE ROBOTIC SURRVELLANCE SYSTEM"** is agenuine work carried out by us, in **B.Tech** ***(Computer Science and Engineering (Cyber Security)*** degree course of **Mohan Babu University, Tirupati** and has not been submitted to any other course or University for the award of any degree by us.

We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea / data / fact / source in our submission. We understand that any violation of the above will cause for disciplinary action by the Institute and can also evoke penal action from the sources which have thus not been properly cited or from whom proper permission has not been taken when needed.

Signature of the students

1.

2.

ABSTRACT

The present generation have lots and lots of technology implementations in various sectors, mainly in service sector the major role is being played by the transport department irrespective of whether it's government service of private service. Many people across the country often travel from one place to other which includes travelling for a long distance as well with in a shorter distance. The people who want to travel for a far by place wants to book the travel they choose in advance so the difficulty in booking confirmation during the journey would not be there. Now here comes the problem in earlier days that for booking any kind of travel people need to consult a travel agency which may not be accessible for all the people in their nearby places, so the ticket booking in advance may be very difficult task.

As the present mobile generation is very much aware of the technologies, we can overcome this problem with the simple online web application which can be able to display all the details of the particular travel booking agency. To make all those details available for the viewer the data should be stored, that is we require a data base to make all these details available in a clear alignment and this data base also will be able to store all the details of the viewers who all surfs through the website.

This database helps in storing different kind of entities, relationships, attributes and tuples of the respective relation, the relations include all the company details, administrator details, customer details, details of different kind of transports available, services details, feedback of the customer, booking details, cancellation details, receipt details, etc. These details many be additionally inserted into the database also some details would be updated and some of them may be deleted as well.

Keywords: SQL Server, Microsoft SQL server management studio, HTML, CSS, java script, PHP server.

TABLE OF CONTENTS

Title	Page No.
ABSTRACT	i
CHAPTER 1. INTRODUCTION	
1.1 Introduction to the topic	1-2
1.2 Problem Statement	1-2
1.3 Objectives	3-3
CHAPTER 2. DATABASE DESIGN	
2.1 List of Attributes, entities and relationship	4-7
2.2 E-R Diagram	8-8
CHAPTER 3. RELATIONAL MODEL	
3.1 Database languages	9-9
3.2 Table Description	10-12
3.3 Relational Database Scheme	13-24
3.4 Relational Queries	25-51
CHAPTER 4. CONCLUSION AND FUTUREWORK	
4.1 Conclusion	52
4.2 Future Work	52

CHAPTER 1. INTRODUCTION

1.1 Introduction to the topic

Remote robotic surveillance uses robots equipped with cameras and sensors to monitor places from a distance. Robots can be useful in locations that are difficult or unsafe for people to inspect, such as factories, forests, disaster areas, and restricted zones. Operators can control robots remotely and use them to carry out missions.

A surveillance operation creates many kinds of information. The system needs to keep details about robots, operators, locations, missions, cameras, and sensors. It may also need to record battery levels, sensor readings, robot movements, communication status, maintenance work, component failures, and mission results.
The Remote Robotic Surveillance System database organizes this information in related tables. This makes it easier to store and retrieve records, connect mission activity to the correct robot and operator, and review the results of surveillance work.

1.2 Problem Statement

Managing information about many robots and missions can become difficult when records are kept separately or entered manually. Users may have trouble finding the robot assigned to a mission, checking its battery or communication status, locating sensor readings, or reviewing maintenance and failure history. Keeping mission reports and recordings separate from their related mission information can also make records harder to manage.

Without a well-organized database, information about robots and surveillance activities may become duplicated, incomplete, or difficult to update. A user may need to search multiple records to find details about one mission. It may also be harder to identify robots that need maintenance or to review incidents from completed missions.

This project addresses these issues by designing a relational database for a remote robotic surveillance system. The database stores information about operators, robots, controls, locations, missions, cameras, sensors, recordings, movements, battery status, communication, maintenance, failures, logs, and reports. Primary and foreign keys connect related records, so users can retrieve information about a mission and the robot, operator, location, and activity associated with it.

1.3 Objectives

The main objective of this project is to design a database that organizes the information needed to manage remote robotic surveillance activities.

The database aims to:

- Store details about robots, operators, locations, and missions.
- Connect each mission with its assigned robot, operator, and location.
- Keep records of camera and sensor equipment, recordings, readings, and robot movements.
- Track battery condition, communication logs, maintenance work, and component failures.
- Store mission logs and reports, including results and incident details.
- Use SQL queries to retrieve, filter, and summarize information from related tables.

The project also aims to create a structure that can be expanded as the surveillance system grows. New robots, operators, locations, and missions can be added while keeping their records connected to the appropriate tables.

Other objectives include making it easier to review robot and mission activity, identify incomplete or failed operations, and retrieve maintenance and equipment information. The database provides a foundation for future features such as a user interface, automatic robot data updates, and alerts for important status changes.

CHAPTER 2. DATABASE DESIGN

2.1 List of Attributes, entities and relationship

1. **Entity Name: Operators**

Attributes	Type
Operator_id	int(10)
Operator_name	varchar(20)
Phone	varchar(13)
Email	varchar(30)
Username	varchar(20)
Role	varchar(10)
Status	varchar(10)

2. **Entity Name: Robots**

Attributes	Type
robot_id	int(10)
robot_name	varchar(25)
robot_model	varchar(25)
serial_number	varchar(50)
current_status	varchar(20)
battery_capacity	varchar(10)

3. **Entity Name: Controls**

Attributes	Type
control_id	INT
operator_id	INT
robot_id	INT
date	Date
start_time	TIME
end_time	TIME
control_mode	VARCHAR(15)
control_status	VARCHAR(15)

4. **Entity Name: Locations**

Attributes	Type
location_id	INT
location_name	VARCHAR(30)
area	VARCHAR(30)
city	VARCHAR(20)
risk_level	INT

accessibility	VARCHAR(20)

5. **Entity Name: Missions**

Attributes	Type
mission_id	INT
mission_name	VARCHAR(30)
mission_type	VARCHAR(25)
robot_id	INT
operator_id	INT
location_id	INT
mission_date	DATE
start_time	TIME
end_time	TIME
mission_status	VARCHAR(15)

6. **Entity Name: Camera**

Attributes	Type
camera_id	INT
robot_id	INT
camera_model	VARCHAR(25)
camera_type	VARCHAR(20)
resolution	VARCHAR(10)
night_vision	VARCHAR(5)
status	VARCHAR(15)

7. **Entity Name: Camera_Settings**

Attributes	Type
setting_id	INT
camera_id	INT
resolution	VARCHAR(10)
night_mode	VARCHAR(10)
zoom_level	INT
recording_mode	VARCHAR(15)

8. **Entity Name: Recordings**

Attributes	Type
recording_id	INT
camera_id	INT
mission_id	INT
recording_date	DATE
start_time	TIME
end_time	TIME

9. **Entity Name: Sensors**

Attributes	Type
sensor_id	INT
robot_id	INT
sensor_type	VARCHAR(20)
sensor_model	VARCHAR(25)
status	VARCHAR(15)

10. **Entity Name: Sensor_Readings**

Attributes	Type
reading_id	INT
sensor_id	INT
mission_id	INT
reading_date	DATE
reading_time	TIME
reading_value	DECIMAL(10,2)
unit	VARCHAR(10)

11. **Entity Name: Robot_Movements**

Attributes	Type
movement_id	INT
robot_id	INT
mission_id	INT
movement_date	DATE
movement_time	TIME
distance	DECIMAL(8,2)
speed	DECIMAL(6,2)

12. **Entity Name: Battery_Status**

Attributes	Type
battery_id	INT
robot_id	INT
mission_id	INT
battery_level	INT
battery_status	VARCHAR(15)
battery_health	VARCHAR(15)

13. **Entity Name: Communication_Logs**

Attributes	Type
communication_id	INT
robot_id	INT
mission_id	INT
communication_date	DATE
connection_type	VARCHAR(15)
communication_status	VARCHAR(15)

14.**Entity Name: Mission_Logs**

Attributes	Type
log_id	INT
mission_id	INT
log_date	DATE
log_time	TIME
event_type	VARCHAR(20)
event_description	VARCHAR(100)

15.**Robot_Status_History**

Attributes	Type
history_id	INT
robot_id	INT
status_date	DATE
status	VARCHAR(20)
remarks	VARCHAR(100)

16. Entity Name: Maintenance

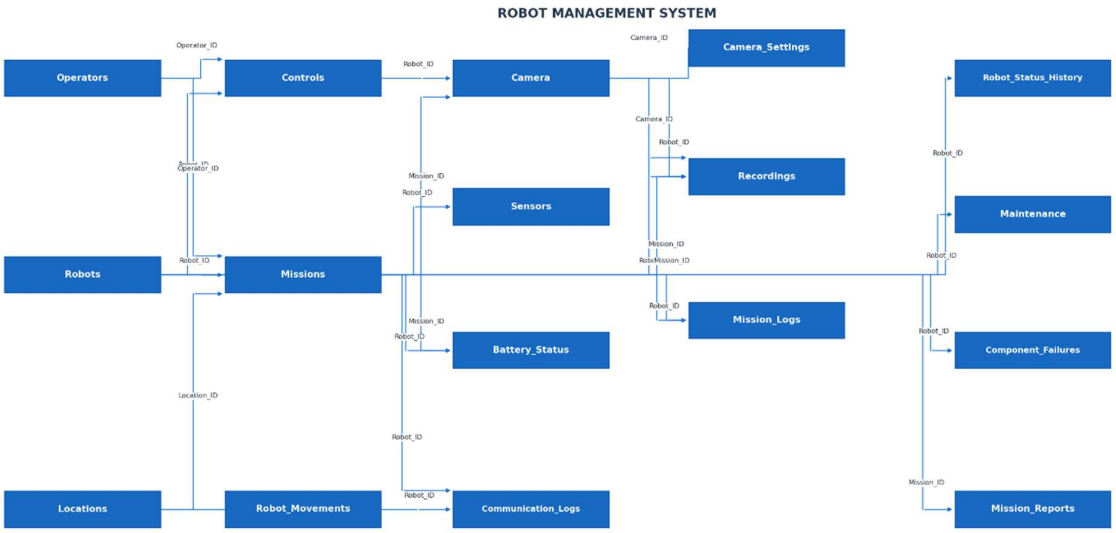
Attributes	Type
maintenance_id	INT
robot_id	INT
maintenance_date	DATE
maintenance_type	VARCHAR(25)
maintenance_status	VARCHAR(15)

17. Entity Name: Component_Failures

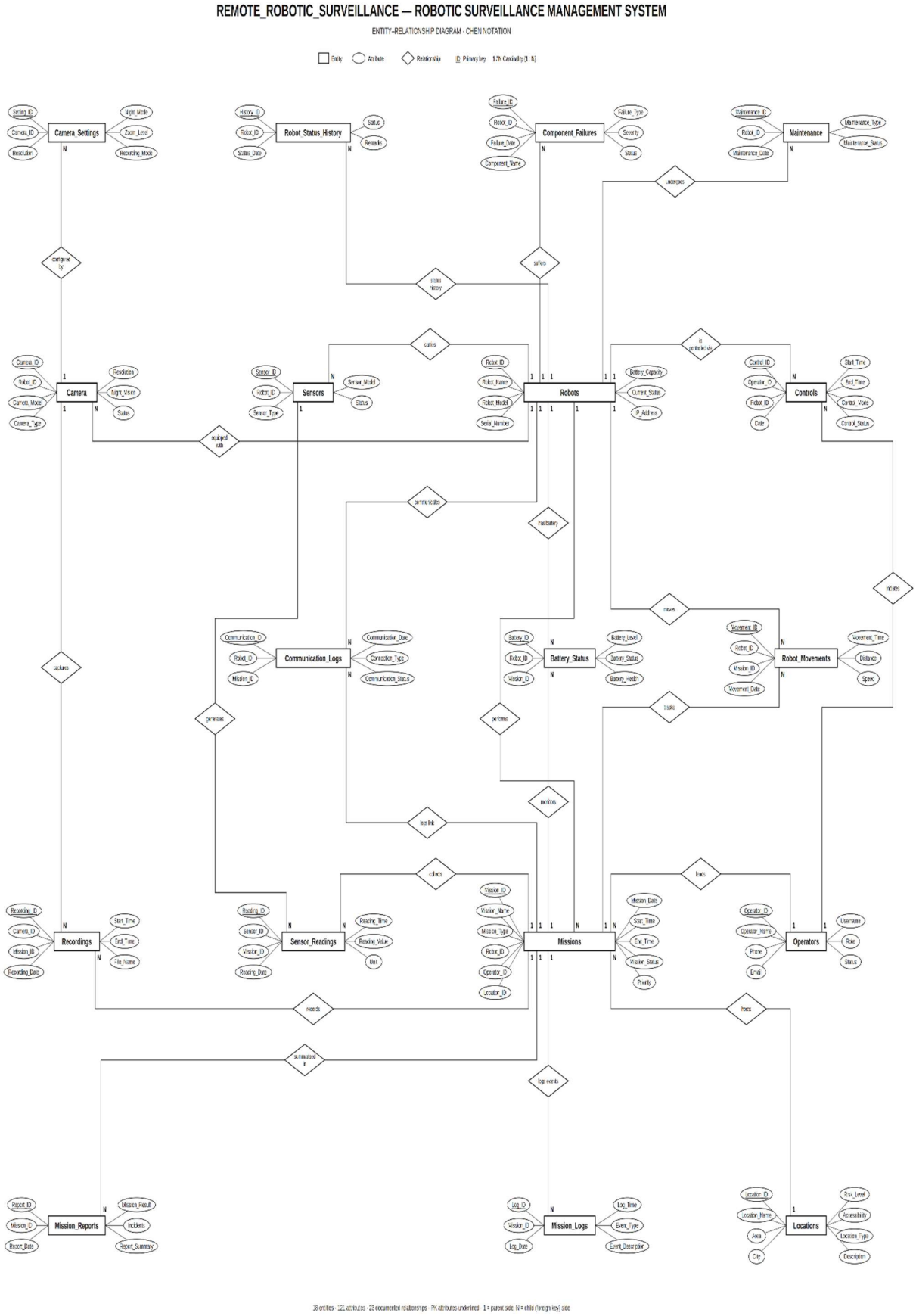
Attributes	Type
failure_id	INT
robot_id	INT
failure_date	DATE
component_name	VARCHAR(25)
failure_type	VARCHAR(30)
severity	INT
status	VARCHAR(15)

18. Entity Name: Mission_Reports

Attributes	Type
report_id	INT
mission_id	INT
report_date	DATE
mission_result	VARCHAR(20)
incidents	INT
report_summary	VARCHAR(150)



2.2 E-R Diagram



CHAPTER 3. RELATIONAL MODEL

3.1. Database languages

Four categories of database languages :

1. Data definition language (DDL)

Data definition language (DDL) creates the framework of the database by specifying the database schema, which is the structure that represents the organization of data. Its common uses include the creation and alteration of tables, files, indexes and columns within the database. This language also allows users to rename or drop the existing database or its components.

Here's a list of DDL statements:

- CREATE: Creates a new database or object, such as a table, index or column.
- ALTER: Changes the structure of the database or object.
- DROP: Deletes the database or existing objects.
- RENAME: Renames the database or existing objects.

2. Data manipulation language (DML)

Data manipulation language (DML) provides operations that handle user requests, offering a way to access and manipulate the data that users store within a database. Its common functions include inserting, updating and retrieving data from the database.

Here's a list of DML statements:

- INSERT: Adds new data to the existing database table.
- UPDATE: Changes or updates values in the table.
- DELETE: Removes records or rows from the table.
- SELECT: Retrieves data from the table or multiple tables.

3. Data control language (DCL)

Data control language (DCL) controls access to the data that users store within a database. Essentially, this language controls the rights and permissions of the database system. It allows users to grant or revoke privileges to the database.

Here's a list of DCL statements:

- GRANT: Gives a user access to the database.
- REVOKE: Removes a user's access to the database.

4. Transaction control language (TCL)

Transaction control language (TCL) manages the transactions within a database. Transactions group a set of related tasks into a single, executable task. All the tasks must succeed in order for

the transaction to work. Here's a list of TCL statements:

3.2 Table Description

Following are the tables along with constraints used in All in one travel booking

database.

The Remote Robotic Surveillance database is made up of tables that store information about the people who operate the robots, the robots themselves, and the missions they perform. Other tables keep records of locations, equipment, sensor readings, communication, maintenance, and mission results. Primary keys identify individual records, while foreign keys connect records in related tables.

1. Operators

The Operators table stores information about people who operate or supervise the robots. It includes each operator's name, phone number, email address, username, role, and account status. The role and status fields can help identify an operator's responsibilities and whether the account is active.

Constraints: Operator_id is the primary key. The operator_id fields in Controls and Missions refer to the operator who controlled a robot or was assigned to a mission.

2. Robots

The Robots table stores the main details of each surveillance robot. Its attributes include the robot's name and model, serial number, battery capacity, current status, and IP address. These details help identify a robot and review its basic condition and network information.

Constraints: robot_id is the primary key, and serial_number is unique. The robot_id fields in related tables link records such as missions, controls, battery records, sensor details, and maintenance to the correct robot.

3. Controls

The Controls table records sessions in which an operator controls a robot. Each record includes the operator, robot, date, start and end times, control mode, and control status. This allows control activity to be associated with both the person operating the system and the robot being controlled.

Constraints: control_id is the primary key. operator_id is a foreign key referencing Operators, and robot_id is a foreign key referencing Robots.

4. Locations

The Locations table contains details about areas monitored by the surveillance system. It stores the location name, area, city, risk level, accessibility, location type, and a description. This information can be used to identify where a mission takes place and to describe the conditions of that area.

Constraints: location_id is the primary key. The location_id field in Missions refers to the location selected for a mission.

5. Missions

The Missions table stores the main information about surveillance tasks. A mission record

includes its name and type, assigned robot and operator, location, date, start and end times, status, and priority. These details help organize missions and connect their activity records, such as logs, recordings, readings, and reports.

Constraints: mission_id is the primary key. robot_id, operator_id, and location_id are foreign keys referencing Robots, Operators, and Locations.

6. Camera

The Camera table stores information about cameras installed on robots. It includes the robot to which the camera belongs, its model and type, resolution, night-vision capability, and status. This information identifies the camera equipment available for surveillance and its current condition.

Constraints: camera_id is the primary key. robot_id is a foreign key referencing Robots.

7. Camera_Settings

The Camera_Settings table stores configuration details for cameras. These include the camera's resolution, night-mode setting, zoom level, and recording mode. Keeping settings in a separate table allows camera configuration information to be recorded independently from the camera's basic details.

Constraints: setting_id is the primary key. camera_id is a foreign key referencing Camera.

8. Recordings

The Recordings table stores information about video files captured during missions. Each record identifies the camera and mission associated with the recording, along with the recording date, start and end times, and file name. These fields help connect a recording to the equipment and mission that produced it.

Constraints: recording_id is the primary key. camera_id is a foreign key referencing Camera, and mission_id is a foreign key referencing Missions.

9. Sensors

The Sensors table stores details about sensors fitted to robots. It records the robot associated with each sensor, the sensor type and model, and its status. This makes it possible to identify the sensors available on each robot and their recorded condition.

Constraints: sensor_id is the primary key. robot_id is a foreign key referencing Robots.

10. Sensor_Readings

The Sensor_Readings table stores values collected by sensors during missions. Each reading is linked to a sensor and a mission and includes the date, time, measured value, and unit. These records can be used to review sensor data in the context of a particular mission.

Constraints: reading_id is the primary key. sensor_id is a foreign key referencing Sensors, and mission_id is a foreign key referencing Missions.

11. Robot_Movements

The Robot_Movements table records movement information for robots during missions. It includes the robot and mission, the movement date and time, distance travelled, and speed. These records help describe the movement activity associated with each mission.

Constraints: movement_id is the primary key. robot_id and mission_id are foreign keys

referencing Robots and Missions.

12. Battery_Status

The Battery_Status table stores battery information for robots and their missions. It includes the robot and mission, battery level, battery status, and battery health. These records can help track battery condition while reviewing robot activity.

Constraints: battery_id is the primary key. robot_id is a foreign key referencing Robots, and mission_id is a foreign key referencing Missions.

13. Communication_Logs

The Communication_Logs table records communication information for robots during missions. It stores the robot and mission, communication date, connection type, and communication status. These entries provide a record of the communication condition associated with robot activity.

Constraints: communication_id is the primary key. robot_id and mission_id are foreign keys referencing Robots and Missions.

14. Mission_Logs

The Mission_Logs table records events that occur during a mission. Each entry includes the mission, log date and time, event type, and a description of the event. Multiple log entries can be associated with the same mission to record its activity over time.

Constraints: log_id is the primary key. mission_id is a foreign key referencing Missions.

15. Robot_Status_History

The Robot_Status_History table keeps a history of recorded robot statuses. Each entry identifies the robot and includes the date of the status record and any remarks. Keeping these entries in a separate table allows multiple status records to be associated with one robot.

Constraints: history_id is the primary key. robot_id is a foreign key referencing Robots.

16. Maintenance

The Maintenance table stores maintenance records for robots. It includes the robot, maintenance date, maintenance type, and maintenance status. These records help keep a history of maintenance work associated with each robot.

Constraints: maintenance_id is the primary key. robot_id is a foreign key referencing Robots.

17. Component_Failures

The Component_Failures table records failures affecting robot components. Its attributes include the robot, failure date, component name, failure type, severity, and status. These details help identify which component was affected and provide information about the recorded failure.

Constraints: failure_id is the primary key. robot_id is a foreign key referencing Robots.

18. Mission_Reports

The Mission_Reports table stores summary information about missions. Each report is linked to a mission and includes the report date, mission result, number of incidents, and a written summary. This provides a record of the reported outcome and any incidents associated with a mission.

Constraints: report_id is the primary key. mission_id is a foreign key referencing Missions.

3.3 Relational Database Scheme

The relational schemes below use the table names and attribute names from the source SQL. PK and FK labels are included to make the relationships easier to identify.

1. Controls(control_id (PK), operator_id (FK), robot_id (FK), date, start_time, end_time, control_mode, control_status)
2. Locations(location_id (PK), location_name, area, city, risk_level, accessibility, location_type, description)
3. Missions(mission_id (PK), mission_name, mission_type, robot_id (FK), operator_id (FK), location_id (FK), mission_date, start_time, end_time, mission_status, priority)
4. Camera(camera_id (PK), robot_id (FK), camera_model, camera_type, resolution, night_vision, status)
5. Camera_Settings(setting_id (PK), camera_id (FK), resolution, night_mode, zoom_level, recording_mode)
6. Recordings(recording_id (PK), camera_id (FK), mission_id (FK), recording_date, start_time, end_time, file_name)
7. Sensors(sensor_id (PK), robot_id (FK), sensor_type, sensor_model, status)
8. Sensor_Readings(reading_id (PK), sensor_id (FK), mission_id (FK), reading_date, reading_time, reading_value, unit)
9. Robot_Movements(movement_id (PK), robot_id (FK), mission_id (FK), movement_date, movement_time, distance, speed)
10. Battery_Status(battery_id (PK), robot_id (FK), mission_id (FK), battery_level, battery_status, battery_health)
11. Communication_Logs(communication_id (PK), robot_id (FK), mission_id (FK), communication_date, connection_type, communication_status)
12. Mission_Logs(log_id (PK), mission_id (FK), log_date, log_time, event_type, event_description)
13. Robot_Status_History(history_id (PK), robot_id (FK), status_date, status, remarks)
14. Maintenance(maintenance_id (PK), robot_id (FK), maintenance_date, maintenance_type, maintenance_status)
15. Component_Failures(failure_id (PK), robot_id (FK), failure_date, component_name, failure_type, severity, status)
16. Mission_Reports(report_id (PK), mission_id (FK), report_date, mission_result, incidents, report_summary)

3.4 Relational Database Tables

The following tables are used in the Remote Robotic Surveillance database. Each section shows the table definition, the records inserted, and the result of selecting all records. PK means Primary Key; FK means Foreign Key.

```
CREATE DATABASE Remote_Robotic_Surveillance
GO
USE Remote_Robotic_Surveillance
GO

/* Create table Operators */
CREATE TABLE Operators(
Operator_id          INT PRIMARY KEY,
Operator_name    VARCHAR(20) NOT NULL,
Phone                VARCHAR(13) NOT NULL,
Email                VARCHAR(30),
Username             VARCHAR(20) NOT NULL,
Role                 VARCHAR(10) NOT NULL,
Status               VARCHAR(10) NOT NULL
)

INSERT INTO Operators VALUES
(101, 'Praveen Kumar', '9876543210', 'praveen7@gmail.com', 'praveen7', 'Operator', 'Active'),
(102, 'Rahul Sharma', '9123456780', 'rahul@gmail.com', 'rahul102', 'Operator', 'Active'),
(103, 'Priyanka', '9988776655', 'priya78@gmail.com', 'priya53', 'Supervisor', 'Active'),
(104, 'Kiran', '9345678120', 'kiran@gmail.com', 'kiran4', 'Operator', 'Active'),
(105, 'Leeladhar', '8765432109', 'lallu2@gmail.com', 'lallu20', 'Operator', 'Inactive'),
(106, 'Vikram Singh', '9098765432', 'vikram@gmail.com', 'vikram6', 'Supervisor', 'Active'),
(107, 'Anjali Devi', '8899776655', 'anjali@gmail.com', 'anjali7', 'Operator', 'Active'),
(108, 'Rohit Verma', '7788990011', 'rohit@gmail.com', 'rohit108', 'Operator', 'Active'),
(109, 'Neelakanta', '9871234560', 'neela@gmail.com', 'neela55', 'Operator', 'Inactive'),
(110, 'Suresh', '9012345678', 'suresh@gmail.com', 'suresh10', 'Admin', 'Active')


/* Create table Robots */
```

REMOTE ROBOTIC SURVEILLANCE SYSTEM

```
CREATE TABLE Robots(
robot_id            INT PRIMARY KEY,
robot_name                  VARCHAR(20) NOT NULL,
robot_model                 VARCHAR(10),
serial_number   INT UNIQUE NOT NULL,
battery_capacity INT NOT NULL,
current_status  VARCHAR(20),
ip_address                  VARCHAR(15) NOT NULL
)

INSERT INTO Robots VALUES
(201, 'Chitti', 'C100', 500001, 12000, 'Available', '192.168.1.101'),
(202, 'NightEye', 'NV200', 500002, 15000, 'On Mission', '192.168.1.102'),
(203, 'Chitti 2.0', 'C200', 500003, 18000, 'Available', '192.168.1.103'),
(204, 'Optimus', 'GS400', 500004, 20000, 'Maintenance', '192.168.234.104'),
(205, 'Jarvis', 'J500', 500005, 14000, 'On Mission', '192.168.1.105'),
(206, 'Vennala', 'SR600', 500006, 16000, 'Available', '192.168.234.106'),
(207, 'MonitorBot', 'NS700', 500007, 18000, 'Offline', '192.168.1.107'),
(208, 'Groot', 'RG800', 500008, 22000, 'On Mission', '192.168.234.108'),
(209, 'Phoenix', 'DB900', 500009, 25000, 'Available', '192.168.235.109'),
(210, 'Hercules', 'PT100', 500010, 13000, 'Maintenance', '192.168.234.110')

/* Create table Controls */
CREATE TABLE Controls(
control_id      INT PRIMARY KEY,
operator_id     INT FOREIGN KEY (operator_id) REFERENCES Operators(operator_id),
robot_id        INT FOREIGN KEY (robot_id) REFERENCES Robots(robot_id),
date                        Date,
start_time      TIME,
end_time        TIME,
control_mode    VARCHAR(15),
control_status  VARCHAR(15)
)

INSERT INTO Controls VALUES
(301, 101, 207, '2026-08-03', '08:15:00', '09:40:00', 'Android', 'Completed'),
(302, 114, 202, '2026-08-07', '14:20:00', '15:10:00', 'Remote', 'Interrupted'),
(303, 106, 213, '2026-08-12', '19:30:00', '21:00:00', 'Android', 'Completed'),
(304, 103, 205, '2026-08-18', '06:45:00', '08:25:00', 'Remote', 'Completed'),
```

```
(305, 109, 201, '2026-08-23', '16:10:00', '17:05:00', 'Android', 'Completed'),
(306, 115, 210, '2026-08-29', '11:35:00', '12:20:00', 'Remote', 'Interrupted'),
(307, 102, 214, '2026-09-02', '09:50:00', '11:30:00', 'Android', 'Completed'),
(308, 111, 203, '2026-09-06', '20:15:00', '22:05:00', 'Remote', 'Completed'),
(309, 105, 209, '2026-09-11', '13:25:00', '14:00:00', 'Android', 'Active'),
(310, 108, 215, '2026-09-16', '07:30:00', '09:15:00', 'Remote', 'Completed')

/* Create table Locations */
CREATE TABLE Locations(
location_id     INT PRIMARY KEY,
location_name   VARCHAR(30) NOT NULL,
area            VARCHAR(30),
city            VARCHAR(20),
risk_level      INT NOT NULL,
accessibility   VARCHAR(20),
location_type   VARCHAR(20),
description     VARCHAR(100)
)

INSERT INTO Locations VALUES
(401, 'Factory Zone A', 'Production Block', 'Hyderabad', 8, 'Restricted', 'Industrial', 'Heavy machinery operating area'),
(402, 'Warehouse Block B', 'Storage Area', 'Bengaluru', 4, 'Open', 'Warehouse', 'Storage and inventory inspection area'),
(403, 'Disaster Zone 1', 'Collapsed Building', 'Chennai', 10, 'Difficult', 'Disaster', 'Damaged building requiring surveillance'),
(404, 'Forest Area North', 'North Sector', 'Coimbatore', 8, 'Difficult', 'Forest', 'Dense forest area'),
(405, 'Construction Site A', 'Building Zone', 'Pune', 7, 'Restricted', 'Construction', 'Active construction site'),
(406, 'Residential Block C', 'Street 4', 'Delhi', 3, 'Open', 'Residential', 'Residential surveillance area'),
(407, 'Security Gate 1', 'Main Entrance', 'Mumbai', 7, 'Restricted', 'Security Zone', 'Main security entrance'),
(408, 'Parking Area East', 'East Wing', 'Kochi', 2, 'Open', 'Parking', 'Vehicle parking area'),
(409, 'Industrial Yard', 'Equipment Section', 'Visakhapatnam', 8, 'Restricted', 'Industrial', 'Industrial equipment storage area'),
(410, 'Rescue Area 2', 'Emergency Zone', 'Bhopal', 10, 'Difficult', 'Disaster', 'Emergency search and rescue location')

/* Create table Missions */
```

```
CREATE TABLE Missions(
mission_id       INT PRIMARY KEY,
mission_name     VARCHAR(30) NOT NULL,
mission_type     VARCHAR(25),
robot_id         INT NOT NULL FOREIGN KEY (robot_id) REFERENCES Robots(robot_id),
operator_id      INT NOT NULL FOREIGN KEY (operator_id) REFERENCES Operators(operator_id),
location_id      INT NOT NULL FOREIGN KEY (location_id) REFERENCES Locations(location_id),
mission_date     DATE,
start_time       TIME,
end_time         TIME,
mission_status   VARCHAR(15),
priority INT CHECK (priority BETWEEN 1 AND 5)
)

INSERT INTO Missions VALUES
(501, 'Night Factory Inspection', 'Inspection', 202, 101, 401, '2026-09-01', '20:00:00', '22:15:00', 'Completed', 4),
(502, 'Warehouse Surveillance', 'Surveillance', 205, 102, 402, '2026-09-03', '09:30:00', '11:10:00', 'Completed', 3),
(503, 'Building Search Operation', 'Search', 209, 103, 403, '2026-09-05', '14:00:00', '16:30:00', 'Completed', 5),
(504, 'Forest Patrol', 'Patrol', 206, 104, 404, '2026-09-07', '06:45:00', '09:00:00', 'Completed', 4),
(505, 'Construction Safety Check', 'Inspection', 214, 106, 405, '2026-09-09', '10:15:00', '12:00:00', 'Completed', 4),
(506, 'Residential Security Patrol', 'Surveillance', 201, 107, 406, '2026-09-11', '18:30:00', '20:00:00', 'Completed', 3),
(507, 'Main Gate Monitoring', 'Security', 208, 108, 407, '2026-09-13', '21:00:00', '23:00:00', 'Completed', 4),
(508, 'Parking Area Inspection', 'Inspection', 215, 109, 408, '2026-09-15', '08:00:00', '09:20:00', 'Completed', 1),
(509, 'Industrial Equipment Survey', 'Inspection', 203, 110, 409, '2026-09-17', '11:30:00', '13:15:00', 'Completed', 4),
(510, 'Emergency Rescue Search', 'Rescue', 209, 111, 410, '2026-09-19', '15:00:00', '18:00:00', 'Completed', 5)

/* Create table Camera */
CREATE TABLE Camera(
camera_id       INT PRIMARY KEY,
robot_id        INT NOT NULL FOREIGN KEY (robot_id) REFERENCES Robots(robot_id),
```

```
camera_model    VARCHAR(25),
camera_type     VARCHAR(20),
resolution      VARCHAR(10),
night_vision    VARCHAR(5),
status          VARCHAR(15)
)

INSERT INTO Camera VALUES
(601, 201, 'GoPro 12', 'Action', '5.3K', 'No', 'Active'),
(602, 202, 'GoPro 11', 'Action', '5.3K', 'No', 'Active'),
(603, 203, 'DJI 4', 'Action', '4K', 'No', 'Active'),
(604, 204, 'Hikvision', 'Security', '4MP', 'Yes', 'Faulty'),
(605, 205, 'GoPro 10', 'Action', '5.3K', 'No', 'Active'),
(606, 206, 'DJI 3', 'Action', '4K', 'No', 'Active'),
(607, 207, 'GoPro 9', 'Action', '5K', 'No', 'Offline'),
(608, 208, 'Hikvision 2CD', 'Security', '4MP', 'Yes', 'Active'),
(609, 209, 'GoPro 12', 'Action', '5.3K', 'No', 'Active'),
(610, 210, 'Hikvision 2CD', 'Security', '4MP', 'Yes', 'Faulty')

/* Create table Camera_Settings */
CREATE TABLE Camera_Settings(
setting_id      INT PRIMARY KEY,
camera_id       INT NOT NULL FOREIGN KEY (camera_id) REFERENCES Camera(camera_id),
resolution      VARCHAR(10),
night_mode      VARCHAR(10),
zoom_level      INT,
recording_mode  VARCHAR(15)
)

INSERT INTO Camera_Settings VALUES
(701, 601, '5.3K', 'Off', 1, 'Continuous'),
(702, 602, '5.3K', 'Off', 2, 'Continuous'),
(703, 603, '4K', 'Off', 1, 'Motion'),
(704, 604, '4MP', 'On', 4, 'Continuous'),
(705, 605, '5.3K', 'Off', 2, 'Motion'),
(706, 606, '4K', 'Off', 1, 'Continuous'),
(707, 607, '5K', 'Off', 2, 'Motion'),
(708, 608, '4MP', 'On', 5, 'Continuous'),
(709, 609, '5.3K', 'Off', 1, 'Continuous')
```

```
/* Create table Recordings */
CREATE TABLE Recordings(
recording_id    INT PRIMARY KEY,
camera_id       INT NOT NULL FOREIGN KEY (camera_id) REFERENCES Camera(camera_id),
mission_id      INT NOT NULL FOREIGN KEY (mission_id) REFERENCES Missions(mission_id),
recording_date  DATE,
start_time      TIME,
end_time        TIME,
file_name       VARCHAR(50)
)

INSERT INTO Recordings VALUES
(801, 602, 501, '2026-09-01', '20:05:00', '22:10:00', 'factory_night.mp4'),
(802, 605, 502, '2026-09-03', '09:35:00', '11:05:00', 'warehouse_cam.mp4'),
(803, 609, 503, '2026-09-05', '14:10:00', '16:20:00', 'building_search.mp4'),
(804, 606, 504, '2026-09-07', '06:50:00', '08:55:00', 'forest_patrol.mp4'),
(805, 614, 505, '2026-09-09', '10:20:00', '11:50:00', 'construction.mp4'),
(806, 601, 506, '2026-09-11', '18:35:00', '19:55:00', 'residential.mp4'),
(807, 608, 507, '2026-09-13', '21:10:00', '22:50:00', 'security_gate.mp4'),
(808, 615, 508, '2026-09-15', '08:05:00', '09:15:00', 'parking_area.mp4'),
(809, 603, 509, '2026-09-17', '11:35:00', '13:10:00', 'industrial.mp4'),
(810, 609, 510, '2026-09-19', '15:10:00', '17:50:00', 'rescue_search.mp4'),

/* Create table Sensors */
CREATE TABLE Sensors(
   sensor_id       INT PRIMARY KEY,
   robot_id        INT NOT NULL FOREIGN KEY (robot_id) REFERENCES Robots(robot_id),
   sensor_type     VARCHAR(20),
   sensor_model    VARCHAR(25),
   status          VARCHAR(15)
)

INSERT INTO Sensors VALUES
(901, 201, 'Temperature', 'DHT22', 'Active'),
(902, 202, 'Gas', 'MQ2', 'Active'),
(903, 203, 'Ultrasonic', 'R4', 'Active'),
(904, 204, 'Motion', 'PIR', 'Faulty'),
(905, 205, 'Distance', 'LX', 'Active'),
```

```
(906, 206, 'Temperature', 'DHT11', 'Active'),
(907, 207, 'Motion', 'PIR', 'Offline'),
(908, 208, 'Gas', 'MQ1', 'Active'),
(909, 209, 'Ultrasonic', 'SR04', 'Active'),
(910, 210, 'Temperature', 'DHT22', 'Faulty')

/* Create table Sensor_Readings */
CREATE TABLE Sensor_Readings(
   reading_id INT PRIMARY KEY,
   sensor_id INT NOT NULL FOREIGN KEY REFERENCES Sensors(sensor_id),
   mission_id INT NOT NULL FOREIGN KEY REFERENCES Missions(mission_id),
   reading_date DATE,
   reading_time TIME,
   reading_value DECIMAL(10,2),
   unit VARCHAR(10)
)
INSERT INTO Sensor_Readings VALUES
(1001, 901, 501, '2026-09-01', '20:20:00', 32.50, 'C'),
(1002, 902, 502, '2026-09-03', '09:50:00', 18.40, 'ppm'),
(1003, 903, 503, '2026-09-05', '14:25:00', 6.80, 'm'),
(1004, 904, 504, '2026-09-07', '07:10:00', 1.00, 'detected'),
(1005, 905, 505, '2026-09-09', '10:40:00', 4.25, 'm'),
(1006, 906, 506, '2026-09-11', '18:55:00', 29.80, 'C'),
(1007, 907, 507, '2026-09-13', '21:25:00', 1.00, 'detected'),
(1008, 908, 508, '2026-09-15', '08:20:00', 12.60, 'ppm'),
(1009, 909, 509, '2026-09-17', '11:55:00', 8.45, 'm'),
(1010, 910, 510, '2026-09-19', '15:30:00', 35.20, 'C')

/* Create table Robot_Movements */
CREATE TABLE Robot_Movements(
   movement_id     INT PRIMARY KEY,
   robot_id        INT NOT NULL FOREIGN KEY (robot_id) REFERENCES Robots(robot_id),
   mission_id      INT NOT NULL FOREIGN KEY (mission_id) REFERENCES Missions(mission_id),
   movement_date   DATE,
   movement_time   TIME,
   distance        DECIMAL(8,2),
   speed           DECIMAL(6,2)
)
```

```
INSERT INTO Robot_Movements VALUES
(1101, 202, 501, '2026-09-01', '20:30:00', 125.50, 2.80),
(1102, 205, 502, '2026-09-03', '10:00:00', 85.75, 1.90),
(1103, 209, 503, '2026-09-05', '14:45:00', 62.40, 1.50),
(1104, 206, 504, '2026-09-07', '07:30:00', 210.80, 3.20),
(1105, 214, 505, '2026-09-09', '10:55:00', 95.25, 2.10),
(1106, 201, 506, '2026-09-11', '19:10:00', 145.60, 2.50),
(1107, 208, 507, '2026-09-13', '21:40:00', 78.35, 1.80),
(1108, 215, 508, '2026-09-15', '08:35:00', 54.90, 1.40),
(1109, 203, 509, '2026-09-17', '12:10:00', 130.45, 2.70),
(1110, 209, 510, '2026-09-19', '16:00:00', 185.70, 2.30)

/* Create table Battery_Status */
CREATE TABLE Battery_Status(
   battery_id      INT PRIMARY KEY,
   robot_id        INT NOT NULL FOREIGN KEY (robot_id) REFERENCES Robots(robot_id),
   mission_id      INT FOREIGN KEY (mission_id) REFERENCES Missions(mission_id),
   battery_level   INT,
   battery_status  VARCHAR(15),
   battery_health  VARCHAR(15)
)
INSERT INTO Battery_Status VALUES
(1201, 202, 501, 82, 'Good', 'Excellent'),
(1202, 205, 502, 74, 'Normal', 'Good'),
(1203, 209, 503, 91, 'Good', 'Excellent'),
(1204, 206, 504, 68, 'Normal', 'Good'),
(1205, 214, 505, 85, 'Good', 'Excellent'),
(1206, 201, 506, 61, 'Normal', 'Good'),
(1207, 208, 507, 55, 'Low', 'Good'),
(1208, 215, 508, 88, 'Good', 'Excellent'),
(1209, 203, 509, 72, 'Normal', 'Good'),
(1210, 209, 510, 64, 'Normal', 'Good')

/* Create table Communication_Logs */
CREATE TABLE Communication_Logs(
   communication_id INT PRIMARY KEY,
   robot_id         INT NOT NULL FOREIGN KEY (robot_id) REFERENCES Robots(robot_id),
   mission_id       INT FOREIGN KEY (mission_id) REFERENCES Missions(mission_id),
   communication_date DATE,
```

```
    connection_type  VARCHAR(15),
    communication_status VARCHAR(15)
)

INSERT INTO Communication_Logs VALUES
(1301, 202, 501, '2026-09-01', 'WiFi', 'Stable'),
(1302, 205, 502, '2026-09-03', 'WiFi', 'Stable'),
(1303, 209, 503, '2026-09-05', 'Radio', 'Stable'),
(1304, 206, 504, '2026-09-07', 'WiFi', 'Interrupted'),
(1305, 214, 505, '2026-09-09', 'Radio', 'Stable'),
(1306, 201, 506, '2026-09-11', 'WiFi', 'Stable'),
(1307, 208, 507, '2026-09-13', 'Radio', 'Stable'),
(1308, 215, 508, '2026-09-15', 'WiFi', 'Weak'),
(1309, 203, 509, '2026-09-17', 'WiFi', 'Stable'),
(1310, 209, 510, '2026-09-19', 'Radio', 'Stable')

/* Create table Mission_Logs */
CREATE TABLE Mission_Logs(
    log_id          INT PRIMARY KEY,
    mission_id      INT NOT NULL FOREIGN KEY (mission_id) REFERENCES Missions(mission_id),
    log_date        DATE,
    log_time        TIME,
    event_type      VARCHAR(20),
    event_description VARCHAR(100)
)

INSERT INTO Mission_Logs VALUES
(1401, 501, '2026-09-01', '20:25:00', 'Started', 'Mission Started'),
(1402, 502, '2026-09-03', '09:55:00', 'Inspection', 'Area Inspected'),
(1403, 503, '2026-09-05', '14:30:00', 'Detection', 'Movement Detected'),
(1404, 504, '2026-09-07', '07:20:00', 'Patrol', 'Forest Covered'),
(1405, 505, '2026-09-09', '10:50:00', 'Inspection', 'Equipment Checked'),
(1406, 506, '2026-09-11', '19:05:00', 'Patrol', 'Area Patrolled'),
(1407, 507, '2026-09-13', '21:35:00', 'Monitoring', 'Gate Monitored'),
(1408, 508, '2026-09-15', '08:30:00', 'Inspection', 'Parking Inspected'),
(1409, 509, '2026-09-17', '12:00:00', 'Survey', 'Equipment Surveyed'),
(1410, 510, '2026-09-19', '15:45:00', 'Rescue', 'Survivor Detected')

/* Create table Robot_Status_History */
```

```
CREATE TABLE Robot_Status_History(
   history_id     INT PRIMARY KEY,
   robot_id       INT NOT NULL FOREIGN KEY (robot_id) REFERENCES Robots(robot_id),
   status_date    DATE,
   status         VARCHAR(20),
   remarks        VARCHAR(100)
)

INSERT INTO Robot_Status_History VALUES
(1501, 201, '2026-09-01', 'Available', 'Ready for operation'),
(1502, 202, '2026-09-01', 'On Mission', 'Factory inspection started'),
(1503, 203, '2026-09-04', 'Available', 'Mission completed'),
(1504, 204, '2026-09-10', 'Maintenance', 'Scheduled maintenance'),
(1505, 205, '2026-09-03', 'On Mission', 'Warehouse surveillance'),
(1506, 206, '2026-09-07', 'Available', 'Forest patrol completed'),
(1507, 207, '2026-09-22', 'Offline', 'Communication lost'),
(1508, 208, '2026-09-13', 'On Mission', 'Security patrol started'),
(1509, 209, '2026-09-19', 'On Mission', 'Rescue mission started'),
(1510, 210, '2026-09-15', 'Maintenance', 'Battery service required')

/* Create table Maintenance */
CREATE TABLE Maintenance(
   maintenance_id  INT PRIMARY KEY,
   robot_id       INT NOT NULL FOREIGN KEY (robot_id) REFERENCES Robots(robot_id),
   maintenance_date DATE,
   maintenance_type VARCHAR(25),
   maintenance_status VARCHAR(15)
)
INSERT INTO Maintenance VALUES
(1601, 204, '2026-08-05', 'Battery Service', 'Completed'),
(1602, 210, '2026-08-12', 'Motor Check', 'Completed'),
(1603, 215, '2026-08-20', 'Camera Service', 'Completed'),
(1604, 207, '2026-08-25', 'Communication Check', 'Completed'),
(1605, 204, '2026-09-10', 'Motor Repair', 'In Progress'),
(1606, 201, '2026-09-15', 'Battery Replacement', 'Completed'),
(1607, 213, '2026-09-18', 'Sensor Check', 'Pending'),
(1608, 215, '2026-09-20', 'Camera Repair', 'Completed'),
(1609, 207, '2026-09-23', 'Network Repair', 'In Progress'),
(1610, 204, '2026-09-28', 'System Inspection', 'Pending')
```

```
/* Create table Component_Failures */
CREATE TABLE Component_Failures(
   failure_id      INT PRIMARY KEY,
   robot_id        INT NOT NULL FOREIGN KEY (robot_id) REFERENCES Robots(robot_id),
   failure_date    DATE,
   component_name  VARCHAR(25),
   failure_type    VARCHAR(30),
   severity        INT CHECK (severity BETWEEN 1 AND 5),
   status          VARCHAR(15)
)

INSERT INTO Component_Failures VALUES
(1701, 204, '2026-08-06', 'Motor', 'Overheating', 4, 'Resolved'),
(1702, 210, '2026-08-13', 'Battery', 'Low Capacity', 3, 'Resolved'),
(1703, 207, '2026-08-26', 'Network Module', 'Connection Loss', 4, 'Open'),
(1704, 215, '2026-09-01', 'Camera', 'Image Failure', 3, 'Resolved'),
(1705, 213, '2026-09-18', 'Sensor', 'Sensor Failure', 4, 'Open'),
(1706, 204, '2026-09-10', 'Motor', 'Motor Failure', 5, 'In Progress'),
(1707, 210, '2026-09-15', 'Battery', 'Battery Failure', 4, 'Resolved'),
(1708, 207, '2026-09-22', 'Network Module', 'Signal Loss', 5, 'Open'),
(1709, 215, '2026-09-20', 'Camera', 'Focus Failure', 2, 'Resolved'),
(1710, 213, '2026-09-27', 'Control Unit', 'System Failure', 5, 'Open')

/* Create table Mission_Reports */
CREATE TABLE Mission_Reports(
   report_id       INT PRIMARY KEY,
   mission_id      INT NOT NULL FOREIGN KEY (mission_id) REFERENCES Missions(mission_id),
   report_date     DATE,
   mission_result  VARCHAR(20),
   incidents       INT,
   report_summary  VARCHAR(150)
)

INSERT INTO Mission_Reports VALUES
(1801, 501, '2026-09-01', 'Successful', 0, 'Factory inspection completed'),
(1802, 502, '2026-09-03', 'Successful', 0, 'Warehouse surveillance completed'),
(1803, 503, '2026-09-05', 'Successful', 1, 'Building search completed'),
(1804, 504, '2026-09-07', 'Successful', 0, 'Forest patrol completed'),
```

(1805, 505, '2026-09-09', 'Successful', 0, 'Safety inspection completed'),

(1806, 506, '2026-09-11', 'Successful', 0, 'Residential patrol completed'),

(1807, 507, '2026-09-13', 'Successful', 0, 'Gate monitoring completed'),

(1808, 508, '2026-09-15', 'Successful', 0, 'Parking inspection completed'),

(1809, 509, '2026-09-17', 'Successful', 1, 'Equipment issue detected'),

(1810, 510, '2026-09-19', 'Successful', 2, 'Rescue search completed')

SQL QUERIES :

Query1: Display operator names and phone numbers

SELECT Operator_name, Phone FROM Operators

Query2: Display robot names and their models.

SELECT robot_name, robot_model FROM Robots

Query3: Display operator names and their roles.

SELECT Operator_name, Role FROM Operators

Query4: Display operator names and their email addresses.

SELECT Operator_name, Email FROM Operators

Query5: Display robot names and their current status.

SELECT robot_name, current_status FROM Robots

Query6: Display robot names and their battery capacity.

SELECT robot_name, battery_capacity FROM Robots

Query7: Display all locations.

SELECT * FROM Locations

Query8: Display location names and cities.

SELECT location_name, city FROM Locations

Query9: Display location names and their risk levels.

SELECT location_name, risk_level FROM Locations

Query10: Display all missions.

SELECT * FROM Missions

Query11: Display mission names and mission types.

SELECT mission_name, mission_type FROM Missions

Query12: Display mission names and their priorities.

SELECT mission_name, priority FROM Missions

Query13: Display mission names and their statuses.

SELECT mission_name, mission_status FROM Missions

Query14: Display all camera details.

SELECT * FROM Camera

Query15: Display camera models and their resolutions.

SELECT camera_model, resolution FROM Camera

Query16: Display camera models and their night vision status.

```
SELECT camera_model, night_vision FROM Camera
```

Query17: Display all sensors.

```
SELECT * FROM Sensors
```

Query18: Display sensor types and their models.

```
SELECT sensor_type, sensor_model FROM Sensors
```

Query19: Display operator usernames and their account status.

```
SELECT Username, Status
FROM Operators
```

Query20: Display active supervisors.

```
SELECT Operator_name, Role, Status
FROM Operators
WHERE Role = 'Supervisor'
  AND Status = 'Active'
```

Query21: Display operators whose email address ends with gmail.com.

```
SELECT Operator_name, Email
FROM Operators
WHERE Email LIKE '%@gmail.com'
```

Query22: Display operators whose phone number starts with 9.

```
SELECT Operator_name, Phone
FROM Operators
WHERE Phone LIKE '9%'
```

Query23: Display the different operator roles.

```
SELECT DISTINCT Role
FROM Operators
```

Query24: Display robot names, serial numbers, and IP addresses.

```
SELECT robot_name, serial_number, ip_address
FROM Robots
```

Query25: Display robots with battery capacity between 15000 and 20000.

```
SELECT robot_name, battery_capacity
FROM Robots
WHERE battery_capacity BETWEEN 15000 AND 20000
```

Query26: Display the different robot status values.

```
SELECT DISTINCT current_status
FROM Robots
```

Query27: Display robot names and models in maintenance.

```
SELECT robot_name, robot_model, current_status FROM Robots
WHERE current_status = 'Maintenance'
```

Query28: Display location names, areas, and location types.

```
SELECT location_name, area, location_type FROM Locations
```

Query29: Display locations that are difficult to access.

```
SELECT location_name, city, accessibility FROM Locations
WHERE accessibility = 'Difficult'
```

Query30: Display the different cities in the Locations table.

```
SELECT DISTINCT city FROM Locations
```

Query31: Display locations with a risk level between 5 and 8.

```
SELECT location_name, risk_level
FROM Locations
WHERE risk_level BETWEEN 5 AND 8
```

Query32: Display mission names, dates, and start and end times.

```
SELECT mission_name, mission_date, start_time, end_time
FROM Missions
```

Query33: Display missions that failed.

```
SELECT mission_name, mission_type, mission_date, mission_status
FROM Missions
WHERE mission_status = 'Failed'
```

Query34: Display rescue missions.

```
SELECT mission_name, mission_type, priority
FROM Missions
WHERE mission_type = 'Rescue'
```

Query35: Display planned missions with their priorities.

```
SELECT mission_name, priority, mission_status
FROM Missions
WHERE mission_status = 'Planned'
```

Query36: Display missions scheduled between September 10 and September 20, 2026.

```
SELECT mission_name, mission_date
FROM Missions
WHERE mission_date BETWEEN '2026-09-10' AND '2026-09-20'
```

Query37: Display missions that started after 6:00 PM.

```
SELECT mission_name, start_time
FROM Missions
WHERE start_time >= '18:00:00'
```

Query38: Display camera types, models, and resolutions.

```
SELECT camera_type, camera_model, resolution FROM Camera
```

Query39: Display cameras that are faulty or offline.

```
SELECT camera_model, camera_type, status FROM Camera
WHERE status IN ('Faulty', 'Offline')
```

Query40: Display cameras that have night vision.

```
SELECT camera_model, resolution, night_vision FROM Camera
WHERE night_vision = 'Yes'
```

Query41: Display camera settings with a zoom level greater than 2.

```
SELECT camera_id, resolution, zoom_level, recording_mode
FROM Camera_Settings
WHERE zoom_level > 2
```

Query42: Display recording file names and recording dates.

```
SELECT file_name, recording_date
FROM Recordings
```

Query43: Display recording file names containing the word “forest”.

```
SELECT file_name, recording_date
FROM Recordings
WHERE file_name LIKE '%forest%'
```

Query44: Display sensor IDs, types, and models.

```
SELECT sensor_id, sensor_type, sensor_model
FROM Sensors
```

Query45: Display sensors that are faulty or offline.

```
SELECT sensor_type, sensor_model, status
FROM Sensors
WHERE status IN ('Faulty', 'Offline')
```

Query46: Display temperature sensors.

```
SELECT sensor_id, sensor_model, status
FROM Sensors
WHERE sensor_type = 'Temperature'
```

Query47: Display sensor reading dates, times, and values.

```
SELECT reading_date, reading_time, reading_value, unit
FROM Sensor_Readings
```

Query48: Display sensor readings measured as detected or not detected.

```
SELECT reading_id, reading_value, unit
FROM Sensor_Readings
WHERE unit = 'detected'
```

Query49: Display sensor readings with values between 10 and 30.

```
SELECT reading_id, reading_value, unit
FROM Sensor_Readings
WHERE reading_value BETWEEN 10 AND 30
```

Query50: Display robot movement dates, times, distances, and speeds.

```
SELECT movement_date, movement_time, distance, speed  FROM Robot_Movements
```

Query51: Display movements where the speed is below 2.

```
SELECT movement_id, speed, distance FROM Robot_Movements
```

```
WHERE speed < 2
```

Query52: Display battery levels and battery health.

```
SELECT battery_level, battery_health
FROM Battery_Status
```

Query53: Display battery records marked as Low or Critical.

```
SELECT robot_id, battery_level, battery_status
FROM Battery_Status
WHERE battery_status IN ('Low', 'Critical')
```

Query54: Display battery records with levels between 50 and 80.

```
SELECT robot_id, battery_level, battery_status
FROM Battery_Status
WHERE battery_level BETWEEN 50 AND 80
```

Query55: Display communication dates and connection types.

```
SELECT communication_date, connection_type
FROM Communication_Logs
```

Query56: Display communication records that use Radio.

```
SELECT robot_id, mission_id, connection_type, communication_status
FROM Communication_Logs
WHERE connection_type = 'Radio'
```

Query57: Display communication records with a Stable status.

```
SELECT robot_id, communication_date, communication_status
FROM Communication_Logs
WHERE communication_status = 'Stable'
```

Query58: Display mission log dates, times, and event types.

```
SELECT log_date, log_time, event_type
FROM Mission_Logs
```

Query59: Display mission logs where an event was recorded as Inspection.

```
SELECT log_id, log_date, event_type, event_description
FROM Mission_Logs
WHERE event_type = 'Inspection'
```

Query60: Display robot status history dates, statuses, and remarks.

```
SELECT status_date, status, remarks
FROM Robot_Status_History
```

Query61: Display robot status history records marked as Offline.

```
SELECT robot_id, status_date, remarks
FROM Robot_Status_History
WHERE status = 'Offline'
```

Query62: Display maintenance dates, types, and statuses.

```
SELECT maintenance_date, maintenance_type, maintenance_status
```

FROM Maintenance

Query63: Display maintenance jobs that are Pending.

SELECT robot_id, maintenance_type, maintenance_date

FROM Maintenance

WHERE maintenance_status = 'Pending'

Query64: Display maintenance jobs whose type contains the word “Camera”.

SELECT robot_id, maintenance_date, maintenance_type

FROM Maintenance

WHERE maintenance_type LIKE '%Camera%'

Query65: Display component failure dates, component names, and statuses.

SELECT failure_date, component_name, status

FROM Component_Failures

Query66: Display component failures with severity 5.

SELECT robot_id, component_name, failure_type, severity

FROM Component_Failures

WHERE severity = 5

Query67: Display mission report dates, results, and summaries.

SELECT report_date, mission_result, report_summary

FROM Mission_Reports

Query68: Display successful mission reports.

SELECT mission_id, report_date, mission_result, incidents

FROM Mission_Reports

WHERE mission_result = 'Successful'

Query69: Display sensor types and their status.

SELECT sensor_type, status FROM Sensors

Query70: Display all sensor readings.

SELECT * FROM Sensor_Readings

Query71: Display reading values and their units.

SELECT reading_value, unit FROM Sensor_Readings

Query72: Display all robot movements.

SELECT * FROM Robot_Movements

Query73: Display movement distance and speed.

SELECT distance, speed FROM Robot_Movements

Query74: Display all battery status records.

SELECT * FROM Battery_Status

Query75: Display battery levels and battery status.

SELECT battery_level, battery_status FROM Battery_Status

Query76: Display all communication logs.

SELECT * FROM Communication_Logs

Query77: Display communication type and status.

```
SELECT connection_type, communication_status FROM Communication_Logs
```

Query78: Display all maintenance records.

```
SELECT * FROM Maintenance
```

Query79: Display sensor IDs and their statuses.

```
SELECT sensor_id, status
FROM Sensors
```

Query80: Display active sensors and their models.

```
SELECT sensor_type, sensor_model
FROM Sensors
WHERE status = 'Active'
```

Query81: Display sensors with a faulty status.

```
SELECT sensor_id, sensor_type, sensor_model
FROM Sensors
WHERE status = 'Faulty'
```

Query82: Display sensors with an offline status.

```
SELECT sensor_id, sensor_type, status
FROM Sensors
WHERE status = 'Offline'
```

Query83: Display sensors that use a PIR model.

```
SELECT sensor_id, sensor_type, sensor_model
FROM Sensors
WHERE sensor_model = 'PIR'
```

Query84: Display sensors whose model name contains “DHT”.

```
SELECT sensor_id, sensor_type, sensor_model
FROM Sensors
WHERE sensor_model LIKE '%DHT%'
```

Query85: Display sensors in ascending order by sensor type.

```
SELECT sensor_id, sensor_type, sensor_model
FROM Sensors
ORDER BY sensor_type ASC
```

Query86: Display sensors in descending order by sensor ID.

```
SELECT sensor_id, sensor_type, status
FROM Sensors
ORDER BY sensor_id DESC
```

Query87: Display sensor types and models for sensors with IDs from 905 to 910.

```
SELECT sensor_id, sensor_type, sensor_model
FROM Sensors
WHERE sensor_id BETWEEN 905 AND 910
```

Query88: Display sensors that are either active or faulty.

```
SELECT sensor_id, sensor_type, status
FROM Sensors
WHERE status IN ('Active', 'Faulty')
```

Query89: Display maintenance type and status.

```
SELECT maintenance_type, maintenance_status FROM Maintenance
```

Query90: Display all component failures.

```
SELECT * FROM Component_Failures
```

Query91: Display failed component names and failure types.

```
SELECT component_name, failure_type FROM Component_Failures
```

Query92: Display component names and severity.

```
SELECT component_name, severity FROM Component_Failures
```

Query93: Display all mission reports.

```
SELECT * FROM Mission_Reports
```

Query94: Display mission results and incidents.

```
SELECT mission_result, incidents FROM Mission_Reports
```

Query95: Display all mission logs.

```
SELECT * FROM Mission_Logs
```

Query96: Display event types and descriptions.

```
SELECT event_type, event_description FROM Mission_Logs
```

Query97: Display all robot status history.

```
SELECT * FROM Robot_Status_History
```

Query98: Display robot status and remarks.

```
SELECT status, remarks FROM Robot_Status_History
```

Query99: Display operators whose role is Operator.

```
SELECT * FROM Operators
WHERE Role = 'Operator'
```

Query100: Display robots that are currently available.

```
SELECT * FROM Robots
WHERE current_status = 'Available'
```

Query101: Display robots that are currently on a mission.

```
SELECT * FROM Robots
WHERE current_status = 'On Mission'
```

Query102: Display robots that are under maintenance.

```
SELECT * FROM Robots
WHERE current_status = 'Maintenance'
```

Query103: Display robots with battery capacity greater than 18000.

```
SELECT * FROM Robots
WHERE battery_capacity > 18000
```

Query104: Display locations with risk level greater than 7.

```
SELECT * FROM Locations
WHERE risk_level > 7
```

Query105: Display locations with risk level less than 5.

```
SELECT * FROM Locations
WHERE risk_level< 5
```

Query106: Display high-priority missions.

```
SELECT * FROM Missions
WHERE priority >= 4
```

Query107: Display completed missions.

```
SELECT * FROM Missions
WHERE mission_status = 'Completed'
```

Query108: Display active missions.

```
SELECT * FROM Missions
WHERE mission_status = 'Active'
```

Query109: Display robot status history records marked as Available.

```
SELECT robot_id, status_date, remarks
FROM Robot_Status_History
WHERE status = 'Available'
```

Query110: Display robot status history records marked as On Mission.

```
SELECT robot_id, status_date, remarks
FROM Robot_Status_History
WHERE status = 'On Mission'
```

Query111: Display status history records dated after September 15, 2026.

```
SELECT robot_id, status, status_date
FROM Robot_Status_History
WHERE status_date > '2026-09-15'
```

Query112: Display active operators who are supervisors.

```
SELECT Operator_name, Role, Status
FROM Operators
WHERE Role = 'Supervisor'
  AND Status = 'Active'
```

Query113: Display operators who are administrators.

```
SELECT Operator_name, Username, Status
FROM Operators
WHERE Role = 'Admin'
```

Query114: Display inactive operators.

```
SELECT Operator_name, Role, Status
FROM Operators
```

```
WHERE Status = 'Inactive'
```

Query115: Display operators whose names begin with A.

```
SELECT Operator_name, Role
FROM Operators
WHERE Operator_name LIKE 'A%'
```

Query116: Display operators whose names contain the letter “a”.

```
SELECT Operator_name, Role
FROM Operators
WHERE Operator_name LIKE '%a%'
```

Query117: Display operators who are supervisors or administrators.

```
SELECT Operator_name, Role, Status
FROM Operators
WHERE Role IN ('Supervisor', 'Admin')
```

Query118: Display operators whose usernames contain a number.

```
SELECT Operator_name, Username
FROM Operators
WHERE Username LIKE '%[0-9]%'
```

Query119: Display active operators in alphabetical order.

```
SELECT Operator_name, Role, Status
FROM Operators
WHERE Status = 'Active'
ORDER BY Operator_name ASC
```

Query120: Display robots that are offline.

```
SELECT robot_name, robot_model, current_status
FROM Robots
WHERE current_status = 'Offline'
```

Query121: Display robots with battery capacity from 13000 to 16000.

```
SELECT robot_name, battery_capacity
FROM Robots
WHERE battery_capacity BETWEEN 13000 AND 16000
```

Query122: Display robots with battery capacity of 20000 or more.

```
SELECT robot_name, battery_capacity
FROM Robots
WHERE battery_capacity >= 20000
```

Query123: Display robots whose serial numbers are between 500005 and 500010.

```
SELECT robot_name, serial_number
FROM Robots
WHERE serial_number BETWEEN 500005 AND 500010
```

Query124: Display robots whose names contain “Bot”.

```
SELECT robot_name, robot_model, current_status
FROM Robots
WHERE robot_name LIKE '%Bot%'
```

Query125: Display robots with an IP address beginning with 192.168.234.

```
SELECT robot_name, ip_address
FROM Robots
WHERE ip_address LIKE '192.168.234.%'
```

Query126: Display available or offline robots.

```
SELECT robot_name, current_status
FROM Robots
WHERE current_status IN ('Available', 'Offline')
```

Query127: Display robots with models beginning with C.

```
SELECT robot_name, robot_model
FROM Robots
WHERE robot_model LIKE 'C%'
```

Query128: Display robots with battery capacity below 15000.

```
SELECT robot_name, battery_capacity
FROM Robots
WHERE battery_capacity < 15000
```

Query129: Display locations with a risk level of 10.

```
SELECT location_name, city, risk_level
FROM Locations
WHERE risk_level = 10
```

Query130: Display locations with a risk level between 5 and 8.

```
SELECT location_name, city, risk_level
FROM Locations
WHERE risk_level BETWEEN 5 AND 8
```

Query131: Display locations that are open to access.

```
SELECT location_name, city, accessibility
FROM Locations
WHERE accessibility = 'Open'
```

Query132: Display industrial locations.

```
SELECT location_name, area, city
FROM Locations
WHERE location_type = 'Industrial'
```

Query133: Display locations in Hyderabad.

```
SELECT location_name, area, risk_level
FROM Locations
WHERE city = 'Hyderabad'
```

Query134: Display locations whose names contain “Forest”.

```
SELECT location_name, city, risk_level
FROM Locations
WHERE location_name LIKE '%Forest%'
```

Query135: Display locations with a risk level of 3 or less.

```
SELECT location_name, city, risk_level
FROM Locations
WHERE risk_level <= 3
```

Query136: Display restricted locations with a risk level of 8 or higher.

```
SELECT location_name, city, accessibility, risk_level
FROM Locations
WHERE accessibility = 'Restricted'
  AND risk_level >= 8
```

Query137: Display missions with a priority of 5.

```
SELECT mission_name, mission_type, priority
FROM Missions
WHERE priority = 5
```

Query138: Display missions with priority 1 or 2.

```
SELECT mission_name, priority, mission_status
FROM Missions
WHERE priority IN (1, 2)
```

Query139: Display missions with priority between 2 and 4.

```
SELECT mission_name, mission_type, priority
FROM Missions
WHERE priority BETWEEN 2 AND 4
```

Query140: Display missions whose type is Inspection.

```
SELECT mission_name, mission_date, mission_status
FROM Missions
WHERE mission_type = 'Inspection'
```

Query141: Display missions whose type is Patrol.

```
SELECT mission_name, mission_date, priority
FROM Missions
WHERE mission_type = 'Patrol'
```

Query142: Display missions whose names contain “Night”.

```
SELECT mission_name, mission_date, mission_status
FROM Missions
WHERE mission_name LIKE '%Night%'
```

Query143: Display missions scheduled before September 15, 2026.

```
SELECT mission_name, mission_date, mission_status
```

```
FROM Missions
WHERE mission_date < '2026-09-15'
```

Query144: Display missions scheduled after September 20, 2026.

```
SELECT mission_name, mission_date, mission_status
FROM Missions
WHERE mission_date > '2026-09-20'
```

Query145: Display missions that are Active or Planned.

```
SELECT mission_name, mission_date, mission_status
FROM Missions
WHERE mission_status IN ('Active', 'Planned')
```

Query146: Display missions that are Failed or Interrupted.

```
SELECT mission_name, mission_date, mission_status
FROM Missions
WHERE mission_status IN ('Failed', 'Interrupted')
```

Query147: Display missions that started before 9:00 AM.

```
SELECT mission_name, start_time, mission_status
FROM Missions
WHERE start_time < '09:00:00'
```

Query148: Display missions that ended after 10:00 PM.

```
SELECT mission_name, end_time, mission_status
FROM Missions
WHERE end_time > '22:00:00'
```

Query149: Display planned missions.

```
SELECT * FROM Missions
WHERE mission_status = 'Planned'
```

Query150: Display cameras with night vision enabled.

```
SELECT * FROM Camera
WHERE night_vision = 'Yes'
```

Query151: Display active sensors.

```
SELECT * FROM Sensors
WHERE status = 'Active'
```

Query152: Display battery levels below 50%.

```
SELECT * FROM Battery_Status
WHERE battery_level < 50
```

Query153: Display batteries with critical status.

```
SELECT * FROM Battery_Status
WHERE battery_status = 'Critical'
```

Query154: Display communication logs where connection was lost.

```
SELECT * FROM Communication_Logs
```

```
WHERE communication_status = 'Lost'
```

Query155: Display maintenance records that are completed.

```
SELECT * FROM Maintenance
WHERE maintenance_status = 'Completed'
```

Query156: Display open component failures.

```
SELECT * FROM Component_Failures
WHERE status = 'Open'
```

Query157: Display critical component failures.

```
SELECT * FROM Component_Failures
WHERE severity = 3
```

Query158: Display mission reports where incidents occurred.

```
SELECT * FROM Mission_Reports
WHERE incidents > 0
```

Query159: Display operators in ascending order of their names.

```
SELECT * FROM Operators
ORDER BY Operator_name
```

Query160: Display robots in ascending order of their names.

```
SELECT * FROM Robots
ORDER BY robot_name
```

Query161: Display robots by battery capacity from highest to lowest.

```
SELECT * FROM Robots
ORDER BY battery_capacity DESC
```

Query162: Display locations by risk level from highest to lowest.

```
SELECT * FROM Locations
ORDER BY risk_level DESC
```

Query163: Display missions by priority from highest to lowest.

```
SELECT * FROM Missions
ORDER BY priority DESC
```

Query164: Display missions by date from newest to oldest.

```
SELECT * FROM Missions
ORDER BY mission_date DESC
```

Query165: Display sensor readings from highest to lowest value.

```
SELECT * FROM Sensor_Readings
ORDER BY unit, reading_value DESC
```

Query166: Display robot movements by distance from highest to lowest.

```
SELECT * FROM Robot_Movements
ORDER BY distance DESC
```

Query167: Display battery records from lowest to highest battery level.

```
SELECT * FROM Battery_Status
```

```
ORDER BY battery_level
```

Query168: Display maintenance records by date from newest to oldest.

```
SELECT * FROM Maintenance
ORDER BY maintenance_date DESC
```

Query169: Count the total number of operators.

```
SELECT COUNT(*) AS Total_Operators FROM Operators
```

Query170: Count the total number of robots.

```
SELECT COUNT(*) AS Total_Robots FROM Robots
```

Query171: Count the total number of missions.

```
SELECT COUNT(*) AS Total_Missions FROM Missions
```

Query172: Count the total number of locations.

```
SELECT COUNT(*) AS Total_Locations FROM Locations
```

Query173: Count the total number of cameras.

```
SELECT COUNT(*) AS Total_Cameras FROM Camera
```

Query174: Count the total number of sensors.

```
SELECT COUNT(*) AS Total_Sensors FROM Sensors
```

Query175: Find the highest robot battery capacity.

```
SELECT MAX(battery_capacity) AS Highest_Battery_Capacity FROM Robots
```

Query176: Find the lowest robot battery capacity.

```
SELECT MIN(battery_capacity) AS Lowest_Battery_Capacity FROM Robots
```

Query177: Find the average robot battery capacity.

```
SELECT AVG(battery_capacity) AS Average_Battery_Capacity FROM Robots
```

Query178: Find the highest temperature reading.

```
SELECT MAX(reading_value) AS Highest_Temperature FROM Sensor_Readings
WHERE unit = 'C'
```

Query179: Show missions with their robot names.

```
SELECT m.mission_name, r.robot_name FROM Missions AS m
INNER JOIN Robots AS r ON m.robot_id = r.robot_id
```

Query180: Show missions with the operator responsible for them.

```
SELECT m.mission_name, o.operator_name FROM Missions AS m
INNER JOIN Operators AS o ON m.operator_id = o.operator_id
```

Query181: Show missions with their location names and cities.

```
SELECT m.mission_name, l.location_name, l.city FROM Missions AS m
INNER JOIN Locations AS l ON m.location_id = l.location_id
```

Query182: Show each mission with its robot name.

```
SELECT m.mission_name, r.robot_name FROM Missions AS m
JOIN Robots AS r ON m.robot_id = r.robot_id
```

Query183: Show each camera with its robot.

```
SELECT r.robot_name, c.camera_model, c.camera_type, c.status FROM Robots AS r
```

JOIN Camera AS c ON r.robot_id = c.robot_id

Query184: Show each sensor with its robot.

SELECT r.robot_name, s.sensor_type, s.sensor_model, s.status FROM Robots AS r

JOIN Sensors AS s ON r.robot_id = s.robot_id

Query185: Show sensor readings with the robot name and sensor type.

SELECT r.robot_name, s.sensor_type, sr.reading_value, sr.unit FROM Sensor_Readings AS sr

JOIN Sensors AS s ON sr.sensor_id = s.sensor_id

JOIN Robots AS r ON s.robot_id = r.robot_id

Query186: Show movement records with the robot name and mission name.

SELECT r.robot_name, m.mission_name, rm.distance, rm.speed FROM Robot_Movements AS rm

JOIN Robots AS r ON rm.robot_id = r.robot_id

JOIN Missions AS m ON rm.mission_id = m.mission_id

Query187: Show battery records with robot and mission names.

SELECT r.robot_name, m.mission_name, b.battery_level, b.battery_status

FROM Battery_Status AS b

JOIN Robots AS r ON b.robot_id = r.robot_id

LEFT JOIN Missions AS m ON b.mission_id = m.mission_id

Query188: Show communication records with robot and mission names.

SELECT r.robot_name, m.mission_name, c.connection_type, c.communication_status FROM Communication_Logs AS c

JOIN Robots AS r ON c.robot_id = r.robot_id

LEFT JOIN Missions AS m ON c.mission_id = m.mission_id

Query189: Show maintenance records with robot names.

SELECT r.robot_name, mt.maintenance_type, mt.maintenance_date, mt.maintenance_status

FROM Maintenance AS mt

JOIN Robots AS r ON mt.robot_id = r.robot_id

Query190: Show component failures with robot names.

SELECT r.robot_name, f.component_name, f.failure_type, f.severity, f.status FROM Component_Failures AS f

JOIN Robots AS r ON f.robot_id = r.robot_id

Query191: Show mission reports with mission names.

SELECT m.mission_name, mr.report_date, mr.mission_result, mr.incidents FROM Mission_Reports AS mr

JOIN Missions AS m ON mr.mission_id = m.mission_id

Query192: Show mission logs with mission names.

SELECT m.mission_name, ml.log_date, ml.log_time, ml.event_type, ml.event_description

FROM Mission_Logs AS ml

JOIN Missions AS m ON ml.mission_id = m.mission_id

Query193: Show status history with robot names.

```
SELECT r.robot_name, h.status_date, h.status, h.remarks
FROM Robot_Status_History AS h
JOIN Robots AS r ON h.robot_id = r.robot_id
```

Query194: Show missions and their reports, including missions without a report.

```
SELECT m.mission_name, mr.mission_result, mr.incidents
FROM Missions AS m
LEFT JOIN Mission_Reports AS mr ON m.mission_id = mr.mission_id
```

Query195: Show operators and the missions assigned to them, including operators with none.

```
SELECT o.operator_name, m.mission_name, m.mission_status
FROM Operators AS o
LEFT JOIN Missions AS m ON o.operator_id = m.operator_id
```

Query196: Show robots and their maintenance records, including robots without maintenance.

```
SELECT r.robot_name, mt.maintenance_type, mt.maintenance_status
FROM Robots AS r
LEFT JOIN Maintenance AS mt ON r.robot_id = mt.robot_id
```

Query197: Show robots with their camera and sensor details.

```
SELECT r.robot_name, c.camera_model, s.sensor_type, s.sensor_model
FROM Robots AS r
LEFT JOIN Camera AS c ON r.robot_id = c.robot_id
LEFT JOIN Sensors AS s ON r.robot_id = s.robot_id
```

Query198: Show high-priority missions and the operator and robot assigned.

```
SELECT m.mission_name, m.priority, o.operator_name, r.robot_name FROM Missions AS m
JOIN Operators AS o ON m.operator_id = o.operator_id
JOIN Robots AS r ON m.robot_id = r.robot_id
WHERE m.priority >= 4
```

Query199: Count missions by status.

```
SELECT mission_status, COUNT(*) AS mission_count FROM Missions
GROUP BY mission_status
```

Query200: Count missions by type.

```
SELECT mission_type, COUNT(*) AS mission_count
FROM Missions
GROUP BY mission_type
```

Query201: Count missions by priority.

```
SELECT priority, COUNT(*) AS mission_count
FROM Missions
GROUP BY priority
ORDER BY priority
```

Query202: Count missions handled by each operator.

```
SELECT o.operator_id, o.operator_name, COUNT(m.mission_id) AS missions_handled
```

FROM Operators AS o

LEFT JOIN Missions AS m ON o.operator_id = m.operator_id

GROUP BY o.operator_id, o.operator_name

Query203: Show operators who handled more than one mission.

SELECT o.operator_name, COUNT(m.mission_id) AS missions_handled

FROM Operators AS o

JOIN Missions AS m ON o.operator_id = m.operator_id

GROUP BY o.operator_id, o.operator_name

HAVING COUNT(m.mission_id) > 1

Query204: Count missions assigned to each robot.

SELECT r.robot_name, COUNT(m.mission_id) AS missions_assigned FROM Robots AS r

LEFT JOIN Missions AS m ON r.robot_id = m.robot_id

GROUP BY r.robot_id, r.robot_name

Query205: Show robots assigned to more than one mission.

SELECT r.robot_name, COUNT(m.mission_id) AS missions_assigned

FROM Robots AS r

JOIN Missions AS m ON r.robot_id = m.robot_id

GROUP BY r.robot_id, r.robot_name

HAVING COUNT(m.mission_id) > 1

Query206: Count missions at each location.

SELECT l.location_name, COUNT(m.mission_id) AS mission_count

FROM Locations AS l

LEFT JOIN Missions AS m ON l.location_id = m.location_id

GROUP BY l.location_id, l.location_name

Query207: Show locations with at least two missions.

SELECT l.location_name, COUNT(m.mission_id) AS mission_count

FROM Locations AS l

JOIN Missions AS m ON l.location_id = m.location_id

GROUP BY l.location_id, l.location_name

HAVING COUNT(m.mission_id) >= 2

Query208: Count mission reports by result.

SELECT mission_result, COUNT(*) AS report_count

FROM Mission_Reports

GROUP BY mission_result

Query209: Find the total incidents reported for each mission result.

SELECT mission_result, SUM(incidents) AS total_incidents

FROM Mission_Reports

GROUP BY mission_result

Query210: Count communication records by status.

```
SELECT communication_status, COUNT(*) AS record_count
FROM Communication_Logs
GROUP BY communication_status
```

Query211: Count communication records by connection type and status.

```
SELECT connection_type, communication_status, COUNT(*) AS record_count
FROM Communication_Logs
GROUP BY connection_type, communication_status
```

Query212: Count maintenance jobs by status.

```
SELECT maintenance_status, COUNT(*) AS job_count
FROM Maintenance
GROUP BY maintenance_status
```

Query213: Show robots with at least two maintenance records.

```
SELECT r.robot_name, COUNT(mt.maintenance_id) AS maintenance_count
FROM Robots AS r
JOIN Maintenance AS mt ON r.robot_id = mt.robot_id
GROUP BY r.robot_id, r.robot_name
HAVING COUNT(mt.maintenance_id) >= 2
```

Query214: Count failures by component name.

```
SELECT component_name, COUNT(*) AS failure_count
FROM Component_Failures
GROUP BY component_name
```

Query215: Show component types with more than one failure.

```
SELECT component_name, COUNT(*) AS failure_count
FROM Component_Failures
GROUP BY component_name
HAVING COUNT(*) > 1
```

Query216: Count mission logs by event type.

```
SELECT event_type, COUNT(*) AS event_count
FROM Mission_Logs
GROUP BY event_type
```

Query217: Find total movement distance for each robot.

```
SELECT r.robot_name, SUM(rm.distance) AS total_distance
FROM Robots AS r
JOIN Robot_Movements AS rm ON r.robot_id = rm.robot_id
GROUP BY r.robot_id, r.robot_name
```

Query218: Show robots whose total recorded movement exceeds 250.

```
SELECT r.robot_name, SUM(rm.distance) AS total_distance
FROM Robots AS r
JOIN Robot_Movements AS rm ON r.robot_id = rm.robot_id
```

```
GROUP BY r.robot_id, r.robot_name
HAVING SUM(rm.distance) > 250
```

Query219: Find robots with battery capacity above the average capacity.

```
SELECT robot_name, battery_capacity
FROM Robots
WHERE battery_capacity > (SELECT AVG(battery_capacity) FROM Robots)
```

Query220: Find missions with priority above the average mission priority.

```
SELECT mission_name, priority
FROM Missions
WHERE priority > (
   SELECT AVG(CAST(priority AS DECIMAL(5,2)))
   FROM Missions )
```

Query221: Find locations whose risk level is above the average risk level.

```
SELECT location_name, city, risk_level
FROM Locations
WHERE risk_level > (
   SELECT AVG(CAST(risk_level AS DECIMAL(5,2)))
   FROM Locations )
```

Query222: Find robots with battery below 50%.

```
SELECT r.robot_name, b.battery_level
FROM Robots AS r
JOIN Battery_Status AS b ON r.robot_id = b.robot_id
WHERE b.battery_level < 50
```

Query223: Find missions assigned to robots currently marked as Available.

```
SELECT mission_name, robot_id, mission_status
FROM Missions
WHERE robot_id IN (
   SELECT robot_id
   FROM Robots
  WHERE current_status = 'Available' )
```

Query224: Find operators who have handled at least one failed mission.

```
SELECT operator_name
FROM Operators
WHERE operator_id IN (
   SELECT operator_id
   FROM Missions
   WHERE mission_status = 'Failed')
```

Query225: Show maintenance jobs that are not completed.

```
SELECT maintenance_id, robot_id, maintenance_type, maintenance_status
```

```
FROM Maintenance
WHERE maintenance_status <> 'Completed'
```

Query226: Find missions at locations in a city with multiple locations.

```
SELECT mission_name, location_id
FROM Missions
WHERE location_id IN (
   SELECT location_id
   FROM Locations
   WHERE city IN (
      SELECT city
      FROM Locations
      GROUP BY city
      HAVING COUNT(*) > 1
   )
)
```

Query227: Find each robot's most recent maintenance date.

```
SELECT r.robot_name, MAX(mt.maintenance_date) AS latest_maintenance_date
FROM Robots AS r
LEFT JOIN Maintenance AS mt ON r.robot_id = mt.robot_id
GROUP BY r.robot_id, r.robot_name
```

Query228: Find missions with more incidents than the average report.

```
SELECT m.mission_name, mr.incidents
FROM Mission_Reports AS mr
JOIN Missions AS m ON mr.mission_id = m.mission_id
WHERE mr.incidents > (
   SELECT AVG(CAST(incidents AS DECIMAL(5,2)))
   FROM Mission_Reports  )
```

Query229: Find robots whose battery level is below the average recorded level.

```
SELECT r.robot_name, b.battery_level
FROM Robots AS r
JOIN Battery_Status AS b ON r.robot_id = b.robot_id
WHERE b.battery_level < (
   SELECT AVG(CAST(battery_level AS DECIMAL(5,2)))
   FROM Battery_Status )
```

Query230: Display all control session records.

```
SELECT * FROM Controls
```

Query231: Display control IDs with their operator and robot IDs.

```
SELECT control_id, operator_id, robot_id
FROM Controls
```

Query232: Display the different control modes.

```
SELECT DISTINCT control_mode
FROM Controls
```

Query233: Display control sessions that used Android mode.

```
SELECT control_id, robot_id, [date], control_status
FROM Controls
WHERE control_mode = 'Android'
```

Query234: Display control sessions that used Remote mode.

```
SELECT control_id, operator_id, [date], control_status
FROM Controls
WHERE control_mode = 'Remote'
```

Query235: Display interrupted control sessions.

```
SELECT control_id, operator_id, robot_id, control_status
FROM Controls
WHERE control_status = 'Interrupted'
```

Query236: Display control sessions recorded after September 1, 2026.

```
SELECT control_id, [date], control_mode
FROM Controls
WHERE [date] > '2026-09-01'
```

Query237: Display control sessions from newest to oldest.

```
SELECT control_id, [date], control_mode
FROM Controls
ORDER BY [date] DESC
```

Query238: Display control sessions that started before noon.

```
SELECT control_id, start_time, end_time
FROM Controls
WHERE start_time < '12:00:00'
```

Query239: Display control sessions that ended after 6:00 PM.

```
SELECT control_id, start_time, end_time
FROM Controls
WHERE end_time > '18:00:00'
```

Query240: Display control sessions that are currently active.

```
SELECT control_id, operator_id, robot_id FROM Controls
WHERE control_status = 'Active'
```

Query241: Display control sessions ordered by control mode.

```
SELECT control_id, control_mode, control_status
FROM Controls
ORDER BY control_mode
```

Query242: Count control sessions for each control mode.

```
SELECT control_mode, COUNT(*) AS session_count
FROM Controls
GROUP BY control_mode
```

Query243: Display all camera settings.

```
SELECT * FROM Camera_Settings
```

Query244: Display camera IDs, zoom levels, and recording modes.

```
SELECT camera_id, zoom_level, recording_mode
FROM Camera_Settings
```

Query245: Display camera settings with night mode enabled.

```
SELECT setting_id, camera_id, night_mode
FROM Camera_Settings
WHERE night_mode = 'On'
```

Query246: Display camera settings with night mode disabled.

```
SELECT setting_id, camera_id, night_mode
FROM Camera_Settings
WHERE night_mode = 'Off'
```

Query247: Display camera settings with 4MP resolution.

```
SELECT setting_id, camera_id, resolution
FROM Camera_Settings
WHERE resolution = '4MP'
```

Query248: Display camera settings that use Motion recording mode.

```
SELECT setting_id, camera_id, recording_mode
FROM Camera_Settings
WHERE recording_mode = 'Motion'
```

Query249: Display camera settings with zoom level 1.

```
SELECT setting_id, camera_id, zoom_level
FROM Camera_Settings
WHERE zoom_level = 1
```

Query250: Display the different recording modes.

```
SELECT DISTINCT recording_mode FROM Camera_Settings
```

Query251: Display recording IDs, camera IDs, mission IDs, and file names.

```
SELECT recording_id, camera_id, mission_id, file_name  FROM Recordings
```

Query252: Display recording files with “night” in the file name.

```
SELECT recording_id, file_name
FROM Recordings
WHERE file_name LIKE '%night%'
```

Query253: Display recordings made after September 15, 2026.

```
SELECT recording_id, file_name, recording_date
FROM Recordings
```

```
WHERE recording_date > '2026-09-15'
```

Query254: Display recordings ordered by their start time.

```
SELECT recording_id, file_name, start_time
FROM Recordings
ORDER BY start_time
```

Query255: Display the different recording dates.

```
SELECT DISTINCT recording_date
FROM Recordings
```

Query256: Display the different units used for sensor readings.

```
SELECT DISTINCT unit
FROM Sensor_Readings
```

Query257: Display sensor readings measured in ppm.

```
SELECT reading_id, reading_value, unit
FROM Sensor_Readings
WHERE unit = 'ppm'
```

Query258: Display sensor readings measured in degrees Celsius.

```
SELECT reading_id, reading_value, unit
FROM Sensor_Readings
WHERE unit = 'C'
```

Query259: Display sensor readings recorded at or after 6:00 PM.

```
SELECT reading_id, reading_time, reading_value
FROM Sensor_Readings
WHERE reading_time >= '18:00:00'
```

Query260: Display sensor readings recorded on September 1, 2026.

```
SELECT reading_id, sensor_id, reading_value, unit
FROM Sensor_Readings
WHERE reading_date = '2026-09-01'
```

Query261: Display robot movements with a distance greater than 100.

```
SELECT movement_id, robot_id, distance
FROM Robot_Movements
WHERE distance > 100
```

Query262: Display robot movements with a speed of at least 2.

```
SELECT movement_id, robot_id, speed
FROM Robot_Movements
WHERE speed >= 2
```

Query263: Display robot movements ordered by date and time.

```
SELECT movement_id, movement_date, movement_time, distance
FROM Robot_Movements
ORDER BY movement_date, movement_time
```

Query264: Display robot movements recorded between September 10 and September 20, 2026.

```
SELECT movement_id, robot_id, movement_date, distance
FROM Robot_Movements
WHERE movement_date BETWEEN '2026-09-10' AND '2026-09-20'
```

Query265: Count the movement records for each robot.

```
SELECT robot_id, COUNT(*) AS movement_count
FROM Robot_Movements
GROUP BY robot_id
```

Query266: Display batteries with Fair health.

```
SELECT battery_id, robot_id, battery_health
FROM Battery_Status
WHERE battery_health = 'Fair'
```

Query267: Display batteries marked Good with Excellent health.

```
SELECT battery_id, robot_id, battery_status, battery_health
FROM Battery_Status
WHERE battery_status = 'Good'
  AND battery_health = 'Excellent'
```

Query268: Find the average recorded battery level.

```
SELECT AVG(CAST(battery_level AS DECIMAL(5,2))) AS average_battery_level
FROM Battery_Status
```

Query269: Find the lowest recorded battery level.

```
SELECT MIN(battery_level) AS lowest_battery_level
FROM Battery_Status
```

Query270: Display the different battery health values.

```
SELECT DISTINCT battery_health
FROM Battery_Statu
```

Query271: Display communication records marked Interrupted.

```
SELECT communication_id, robot_id, communication_status
FROM Communication_Logs
WHERE communication_status = 'Interrupted'
```

Query272: Display communication records marked Weak.

```
SELECT communication_id, robot_id, communication_status
FROM Communication_Logs
WHERE communication_status = 'Weak'
```

Query273: Display the different communication connection types.

```
SELECT DISTINCT connection_type
FROM Communication_Logs
```

Query274: Display communication records after September 15, 2026.

```
SELECT communication_id, communication_date, communication_status
```

```
FROM Communication_Logs
WHERE communication_date > '2026-09-15'
```

Query275: Count communication records for each date.

```
SELECT communication_date, COUNT(*) AS record_count
FROM Communication_Logs
GROUP BY communication_date
```

Query276: Display mission log entries where the event type is Started.

```
SELECT log_id, mission_id, log_date, event_description
FROM Mission_Logs
WHERE event_type = 'Started'
```

Query277: Display mission log entries where the event type is Patrol.

```
SELECT log_id, mission_id, log_date, event_description
FROM Mission_Logs
WHERE event_type = 'Patrol'
```

Query278: Display mission logs mentioning a survivor.

```
SELECT log_id, mission_id, event_type, event_description
FROM Mission_Logs
WHERE event_description LIKE '%Survivor%'
```

Query279: Display mission logs dated between September 10 and September 20, 2026.

```
SELECT log_id, mission_id, log_date, event_type
FROM Mission_Logs
WHERE log_date BETWEEN '2026-09-10' AND '2026-09-20'
```

Query280: Display maintenance records for robot 204.

```
SELECT maintenance_id, robot_id, maintenance_date, maintenance_type
FROM Maintenance
WHERE robot_id = 204
```

Query281: Display maintenance records dated after October 1, 2026.

```
SELECT maintenance_id, robot_id, maintenance_date
FROM Maintenance
WHERE maintenance_date > '2026-10-01'
```

Query282: Display maintenance records for Motor Check.

```
SELECT maintenance_id, robot_id, maintenance_date, maintenance_status
FROM Maintenance
WHERE maintenance_type = 'Motor Check'
```

Query283: Display the different maintenance types.

```
SELECT DISTINCT maintenance_type
FROM Maintenance
```

Query284: Display maintenance records ordered by maintenance type.

```
SELECT maintenance_id, maintenance_type, maintenance_status
```

```
FROM Maintenance
ORDER BY maintenance_type
```

Query285: Display component failures with severity from 2 to 4.

```
SELECT failure_id, robot_id, component_name, severity
FROM Component_Failures
WHERE severity BETWEEN 2 AND 4
```

Query286: Display component failures recorded on or after September 15, 2026.

```
SELECT failure_id, component_name, failure_date
FROM Component_Failures
WHERE failure_date >= '2026-09-15'
```

Query287: Display component failures caused by overheating.

```
SELECT failure_id, robot_id, component_name, failure_type
FROM Component_Failures
WHERE failure_type = 'Overheating'
```

Query288: Display component failures ordered by severity from lowest to highest.

```
SELECT failure_id, component_name, severity
FROM Component_Failures
ORDER BY severity ASC
```

Query289: Display mission reports with no reported incidents.

```
SELECT report_id, mission_id, mission_result, incidents
FROM Mission_Reports
WHERE incidents = 0
```

Query290: Display mission reports with a Failed result.

```
SELECT report_id, mission_id, report_date, mission_result
FROM Mission_Reports
WHERE mission_result = 'Failed'
```

Query291: Display mission reports whose summary mentions a rescue.

```
SELECT report_id, mission_id, report_summary
FROM Mission_Reports
WHERE report_summary LIKE '%rescue%'
```

Query292: Display mission reports from oldest to newest.

```
SELECT report_id, mission_id, report_date, mission_result
FROM Mission_Reports
ORDER BY report_date ASC
```

Query293: Display control sessions with operator names and robot names.

```
SELECT c.control_id, o.Operator_name, r.robot_name, c.control_mode
FROM Controls AS c
JOIN Operators AS o ON c.operator_id = o.Operator_id
JOIN Robots AS r ON c.robot_id = r.robot_id
```

Query294: Display camera models with their camera setting modes.

```
SELECT c.camera_model, cs.night_mode, cs.recording_mode
FROM Camera AS c
JOIN Camera_Settings AS cs ON c.camera_id = cs.camera_id
```

Query295: Display recording files with their mission names and camera IDs.

```
SELECT rec.file_name, m.mission_name, rec.camera_id
FROM Recordings AS rec
JOIN Missions AS m ON rec.mission_id = m.mission_id
```

Query296: Display sensor readings with their mission names and sensor IDs.

```
SELECT m.mission_name, sr.sensor_id, sr.reading_value, sr.unit
FROM Sensor_Readings AS sr
JOIN Missions AS m ON sr.mission_id = m.mission_id
```

Query297: Display mission logs with the mission date and event description.

```
SELECT m.mission_name, m.mission_date, ml.event_type, ml.event_description
FROM Mission_Logs AS ml
JOIN Missions AS m ON ml.mission_id = m.mission_id
```

Query298: Display mission reports with the mission type and result.

```
SELECT m.mission_name, m.mission_type, mr.mission_result, mr.incidents
FROM Mission_Reports AS mr
JOIN Missions AS m ON mr.mission_id = m.mission_id
```

Query299: Display component failures with robot models and serial numbers.

```
SELECT r.robot_model, r.serial_number, f.component_name, f.failure_type
FROM Component_Failures AS f
JOIN Robots AS r ON f.robot_id = r.robot_id
```

Query300: Display maintenance records with robot models and serial numbers.

```
SELECT r.robot_model, r.serial_number, mt.maintenance_type, mt.maintenance_status
FROM Maintenance AS mt
JOIN Robots AS r ON mt.robot_id = r.robot_id
```

CHAPTER 4. CONCLUSION AND FUTUREWORK

4.1 Conclusion

The Remote Robotic Surveillance System database provides an organized way to store and manage information about robots, operators, missions, locations, and surveillance equipment. It also keeps records of robot movements, battery status, sensor readings, camera recordings, communication logs, maintenance, component failures, and mission reports.

By linking these records through primary and foreign keys, the database helps users retrieve related information and review robot and mission activity. SQL queries can be used to check robot availability, find missions and their outcomes, monitor maintenance records, and summarize incidents or sensor readings. Overall, the project demonstrates how a relational database can support the management and review of remote robotic surveillance operations.

4.2 Future Work

The Remote Robotic Surveillance System can be improved by connecting the database to a user-friendly application. Operators could use it to view mission, check robot status and access camera or sensor information.

In the future, robot data such as battery level, location, movement and sensor reading could be updated automatically. The system could also send the alerts when a robot has low battery, looses communication or needs maintenance. Adding maps ,user access control and detailed activity reports would make the system more useful for mapping surveillance operation.